

SORBONNE UNIVERSITÉ – PC3R

Mini-projet Web

Forum X Discord

Plateforme de discussion hybride Web / Discord

Étudiant CHEN Florent (21101813)
Encadrant Romain Demangeon

Table des matières

1	Présentation du projet	2
2	Description de l'API Discord	2
2.1	Requêtes utilisées	2
3	Fonctionnalités de l'application	3
3.1	Gestion des utilisateurs	3
3.2	Système de forum	3
3.3	Synchronisation avec Discord	3
3.4	Trace de fonctionnalité implémenter	4
4	Cas d'utilisation classiques	4
4.1	Cas 1 : création d'un sujet depuis le Web vers Discord	4
4.2	Cas 2 : réponse depuis Discord vers le Forum	4
5	Base de données	5
6	Mise à jour des données et appels à l'API externe	6
7	Description du serveur (Back-end Go)	7
7.1	Choix de l'approche	7
7.2	Composition du serveur	7
7.3	Fonctionnalités associées	8
8	Description du client (Front-end React)	8
8.1	Plan du site	8
8.2	Contenu des écrans	9
8.3	Appels au serveur	9
9	Description des requêtes et réponses	11
10	Schéma global du système	12
11	Limites de conception, Problématiques et Solutions	12
11.1	Problèmes liés à la conception actuelle	12
11.2	Analyse des solutions et arbitrages	13
11.3	Perspectives : Protection et Optimisation de la Base de Données	13
12	Utilisation de l'Intelligence Artificielle	14

1. Présentation du projet

Forum X Discord, est une plateforme de discussion web hybride qui permet de centraliser les échanges d'une communauté. L'application offre une interface de forum classique (création de sujets, réponses, profils utilisateurs) tout en étant connectée en temps pseudo réel à un serveur discord. L'objectif est d'unifier les échanges en permettant aux utilisateurs du forums de communiquer directement avec ceux qui utilisent Discord : tout message posté sur le forum apparaît sur le discord et inversement.

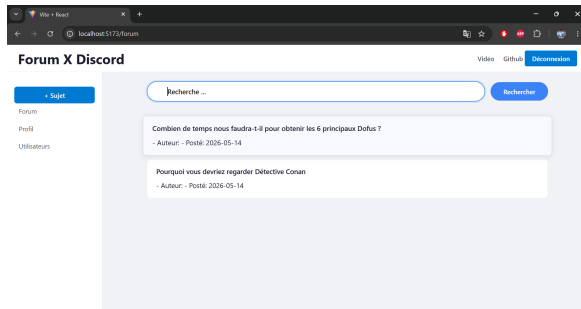


FIGURE 1 – Interface du Forum Web

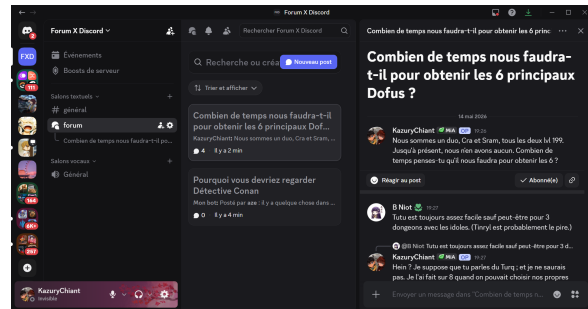


FIGURE 2 – Interface du serveur Discord

2. Description de l'API Discord

Notre application utilise l'API REST de Discord <https://discord.com/api/v10> pour échanger des données aux format json. Cette API offre un accès détaillé à la structure des serveurs Discord : la structure des serveurs (guilds), la hiérarchie des salons (channels), le contenu des messages, ainsi que les informations des utilisateurs et leurs permissions.

2.1. Requêtes utilisées

Notre application exploite spécifiquement quatre types de ressources :

GET /users/@me Identification du bot : valide le token et récupère son identité au lancement.

GET|POST /guilds/{id}/channels Salons :

- en mode **GET** : Il permet d'obtenir l'ensemble des objets "salons". Cela nous permet de vérifier l'existence du salon spécifique.
- en mode **POST** : Si le salon n'est pas trouvé dans l'environnement (Discord) alors on crée dynamiquement le salon.

GET|POST /channels/{id}/threads Fil de discussion :

- en mode **GET** : Il permet de récupérer les fils de discussion sur Discord afin de les recréer sur le site web.
- en mode **POST** : Il permet de créer un nouveau fil de discussion sur le Discord lorsqu'un utilisateur crée un sujet sur le forum web.

GET|POST /channels/{id}/messages Messagerie :

- en mode **GET** : Pour récupérer les messages de Discord, on utilise le paramètre `after={last_id}`. Plutôt que de télécharger l'intégralité des discussions à chaque fois, le programme recherche uniquement les nouveaux messages apparus depuis le dernier "instant" enregistré. Cette méthode provient de l'opérateur `pre` de Lustre, il garantit une synchronisation déterministe et cohérente entre les deux plateformes.
- en mode **POST** : permet d'envoyer les messages du forum vers Discord.

Pour l'ensemble de ces opérations, le programme compare les données du discord avec les données de la base MongoDB. Cela permet de ne traiter que les événements non enregistrés et d'éviter toute duplication d'information.

3. Fonctionnalités de l'application

3.1. Gestion des utilisateurs

- **Inscription et connexion** : le serveur Go gère la création de comptes en stockant les informations dans une base de données MongoDB.
- **Liste d'utilisateur et barre de recherche** : l'application propose une liste des membres inscrits sur le forum. Et une barre de recherche dynamique permet de filtrer les utilisateurs.
- **Profil** : chaque utilisateur possède une page de profil avec ses informations personnelles.

3.2. Système de forum

- **Sujets et Discussions** : On peut créer des sujets pour discuter. Chaque sujet sur le site est lié à un "fil" sur Discord. Si on écrit sur l'un, ça s'affiche sur l'autre.
- **De Discord vers le site** : Le programme regarde régulièrement s'il y a des nouveaux messages sur Discord. S'il en trouve, il les ajoute sur le site.
- **Gestion des messages et des fils** : un utilisateur du site a la possibilité de supprimer ses propres messages ou de supprimer un fil de discussion complet s'il en est l'auteur. Cette action est également répercutée sur le Discord.
- **Barre de recherche** : Il permet de retrouver un fil de discussion rapidement.
- **Admin** : Il a la possibilité de supprimer un fil de discussion ou message même si il ne provient pas de lui.

3.3. Synchronisation avec Discord

- **Du site vers Discord** : Quand un utilisateur poste un message sur le forum, il est envoyé instantanément sur Discord.
- **Système de commentaires** : L'application permet de répondre à un sujet ou de répondre directement à un commentaire.

- **Gestion des messages et des fils** : Le programme vérifie à chaque fois dans la base de données si le message existe déjà pour éviter de l'enregistrer deux fois.

3.4. Trace de fonctionnalité implémenter

- **Filtrage des fils "Administrateur" (Front-end)** : Côté du site web, une logique d'affichage a été implémentée pour rendre certains fils de discussion (*thread/sujet*) invisibles aux yeux des utilisateurs standards. Et seulement les administrateurs peuvent y accéder.
- **Limite de la synchronisation Back-end** : Actuellement, le moteur de synchronisation (Back-end) ne fait pas encore la distinction entre ces fils privés et les fils publics. Par conséquent, tout sujet créé sur le site est automatiquement recréé sur Discord et devient visible par tous.

4. Cas d'utilisation classiques

4.1. Cas 1 : création d'un sujet depuis le Web vers Discord

Un utilisateur se connecte sur le forum et crée un nouveau fil de discussion (*thread/sujet*).

1. **Côté serveur** : le serveur Go reçoit la requête et crée une entrée dans la collection ForumDB (Sujet) de la base de données.
2. **Côté API** : le serveur Go utilise l'API Discord (POST) pour créer dynamiquement un fil de discussion (*thread/sujet*) correspondant sur Discord via le bot.
3. **Résultat** : les membres présents sur Discord voient le message et peuvent y répondre directement depuis Discord.

4.2. Cas 2 : réponse depuis Discord vers le Forum

Un utilisateur Discord voit le fil de discussion (*thread/sujet*) et poste un commentaire en réponse.

1. **Côté serveur** : le serveur Go, via une vérification toutes les 5 à 30 secondes, détecte le nouveau message.
2. **Côté base de données** : Go vérifie que le message n'existe pas encore, puis l'enregistre en le liant au sujet.
3. **Résultat** : la réponse venant de Discord s'affiche dans le fil de discussion du forum.

5. Base de données

Champ	Type	Description
_id	ObjectId	Identifiant unique généré par MongoDB
pseudo	String	Nom d'utilisateur
mdp	String	Mot de passe
date	String	Date d'inscription
rang	String	Rôle de l'utilisateur (ex : admin, membre)

TABLE 1 – Contenu de la collection user

Champ	Type	Description
_id	String	Identifiant unique du sujet généré par Discord
titre	String	Titre du sujet
description	String	Contenu textuel principal
date	String	Date de création du sujet
user_id	String	Identifiant de l'auteur du sujet (clé étrangère vers la collection User)
user_pseudo	String	Pseudonyme de l'auteur du sujet
prive	Bool	Indique si le sujet est privé (true) ou public (false)

TABLE 2 – Structure de la collection ForumDB (Répertoire des sujets/threads)

Champ	Type	Description
_id	String	Identifiant unique du message généré par Discord
sujet_id	String	identifiant du sujet (clé étrangère vers la collection ForumDB)
context	String	Contenu textuel principal
date	String	Date de création du message
user_id	String	Identifiant unique de l'auteur du commentaire (clé étrangère vers la collection user).
user_pseudo	String	Pseudonyme de l'auteur du commentaire
prive	Bool	Indique si le sujet est privé (true) ou public (false)
repond	[]Repond_db	Liste structurée des réponses associées à ce commentaires

TABLE 3 – Structure de la collection ThreadDB (Répertoire des commentaires)

Champ	Type	Description
_id	ObjectId	Identifiant unique généré par MongoDB.
Msg_id	String	Identifiant du message (clé étrangère vers la collection ThreadDB)
Thread_id	String	identifiant du sujet (clé étrangère vers la collection ForumDB)

TABLE 4 – Contenu de la collection DiscordMessages (Derniers index de synchronisation)

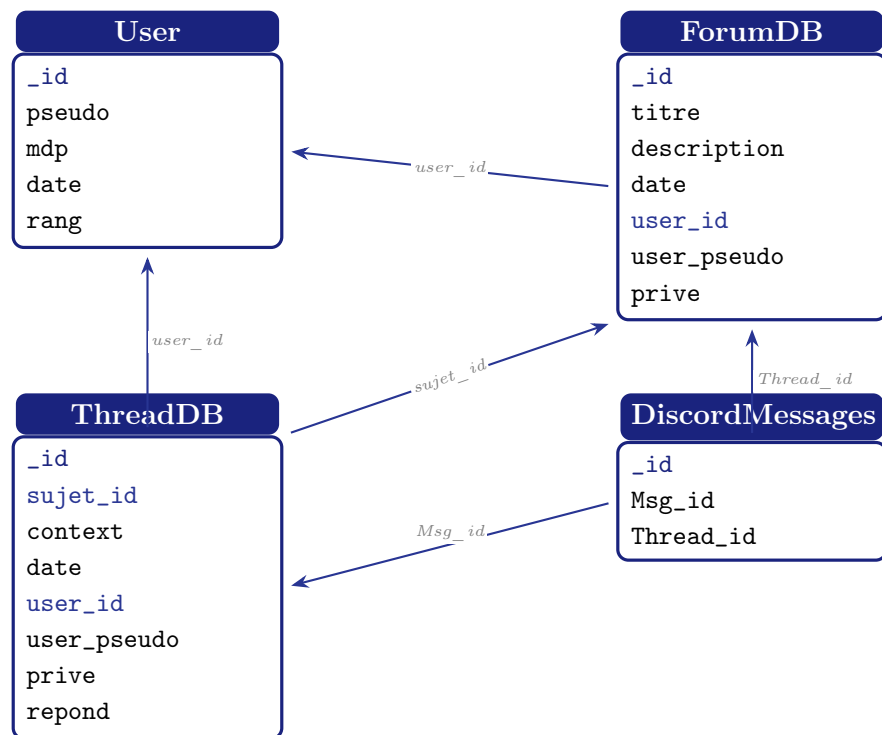


FIGURE 3 – Schéma relationnel

6. Mise à jour des données et appels à l'API externe

Le serveur assure la cohérence des données entre le forum et Discord via trois mécanismes distincts par un **bot Discord** faisant office de passerelle :

- **Flux Discord vers Web (Lecture)** : Une *goroutine* effectue une requête **GET** toutes les 5 à 30 secondes. Elle exploite l'identifiant stocké dans la collection **DiscordMessages** pour ne récupérer que les nouveaux messages via le paramètre **after**. Ces derniers sont ensuite comparés puis intégrés à la collection **ThreadDB** ou **ForumDB** si le sujet n'est pas encore créé.
- **Flux Web vers Discord (Écriture)** : Contrairement à la lecture, l'écriture est **événementielle**. Dès qu'un utilisateur publie un sujet ou un commentaire sur le forum, le serveur déclenche un appel **POST** immédiat vers l'API Discord. Le bot recrée alors instantanément le contenu sur le serveur.

- **Flux Web vers Discord (Suppression)** : Également **événementiel**, lorsqu'un utilisateur supprime un message ou un sujet sur le forum, le serveur envoie une requête `DELETE` à l'API Discord. Cela permet de supprimer le message ou le fil de discussion correspondant sur le serveur discord.

L'implémentation de ces flux, prend compte des **limites de débit** (*Rate Limits*) imposées par l'API Discord. Le choix d'un rafraîchissement périodiquement pour la lecture, plutôt qu'un appel constant, permet d'éviter une saturation temporaire du bot tout en garantissant une synchronisation fiable des données.

7. Description du serveur (Back-end Go)

Le serveur est écrit en Go à l'aide du package `net/http`, utilise le routeur `mux` pour structurer une **architecture en couches** (handler, service et repositories).

7.1. Choix de l'approche

Le serveur adopte une architecture **handler** / **service** / **repository** :

- les **handlers** reçoivent les requêtes HTTP et renvoient les réponses JSON ;
- les **services** appliquent les règles métier et gèrent les échanges entre le forum, MongoDB et Discord ;
- les **repositories** assurent uniquement la persistance et la lecture des données dans MongoDB ;
- les **middlewares** gèrent les traitements transversaux (CORS, authentification, journalisation).

7.2. Composition du serveur

- `main.go` : point d'entrée du serveur. Il initialise la connexion MongoDB, crée les repositories, services et handlers, configure le bot Discord, démarre la synchronisation et enregistre toutes les routes HTTP.
- `handler/user_handler.go` : gère les requêtes liées aux utilisateurs (connexion, inscription, déconnexion, récupération du profil courant, liste des utilisateurs, recherche par identifiant).
- `handler/forum_handler.go` : gère les requêtes liées au forum (création et suppression des sujets, création et suppression des messages, récupération des fils, flux temps réel SSE).
- `service/user.go` : contient la logique métier des utilisateurs, notamment la vérification des identifiants, l'inscription et la récupération des données utilisateur.
- `service/forum.go` : contient la logique métier du forum, avec la création, la lecture et la suppression des sujets et des messages.

- `service/discord.go` et `service/discord_api.go` : assurent la passerelle vers Discord, la création des threads, l'envoi des messages, la récupération des nouveaux messages et la suppression côté Discord.
- `service/forum_events.go` : implémente un hub d'événements publié aux clients connectés en SSE afin de mettre à jour l'interface en temps réel.
- `repository/user_db.go` : encapsule l'accès MongoDB pour les utilisateurs.
- `repository/forum_db.go` : encapsule l'accès MongoDB pour les sujets, les messages et les réponses.
- `repository/discord_db.go` : stocke l'identifiant du dernier message traité pour chaque thread Discord, afin d'éviter les doublons lors de la synchronisation.
- `middleware/authentication.go` et `middleware/main_middleware.go` : assurent la vérification des requêtes, la gestion des cookies de session, le CORS et le logging.

7.3. Fonctionnalités associées

- **Authentification** : le serveur reçoit les identifiants, vérifie leur validité dans MongoDB, crée une session et autorise l'accès aux routes protégées.
- **Gestion du forum** : les utilisateurs peuvent créer des sujets, consulter les fils, poster des messages, répondre à un message existant et supprimer leur contenu si les droits le permettent.
- **Synchronisation avec Discord** : dès qu'un sujet ou un message est créé sur le forum, le serveur crée l'élément correspondant sur Discord. Inversement, une goroutine interroge périodiquement Discord pour récupérer les nouveaux messages et les réinjecter dans le forum.
- **Mise à jour en temps réel** : un flux SSE permet au client de recevoir immédiatement les événements de création, mise à jour ou suppression sans recharger la page.
- **Protection et qualité de service** : les middlewares ajoutent le CORS, le contrôle des requêtes vides et la journalisation, tandis que les timeouts applicatifs limitent les blocages lors des appels réseau.

8. Description du client (Front-end React)

La partie client est écrite en **React**. La navigation est gérée par `react-router-dom` : `App.jsx` vérifie d'abord la session avec `GET /api/user/me`, puis affiche soit l'espace d'authentification, soit le tableau de bord principal.

8.1. Plan du site

- **Espace d'authentification** : `/login` et `/register`.
- **Accueil du forum** : `/forum`, avec la liste des sujets.
- **Fil de discussion** : `/forum/sujet`, qui affiche un sujet, ses commentaires et le for-

mulaire de réponse.

- **Création d'un sujet** : /postforum.
- **Profil utilisateur** : /profile.
- **Liste des utilisateurs** : /listuser.

8.2. Contenu des écrans

- **Connexion** (Login.jsx) : formulaire de connexion avec pseudo et mot de passe.
- **Inscription** (Register.jsx) : formulaire d'inscription avec confirmation du mot de passe.
- **Accueil forum** (Forum.jsx) : affichage de tous les sujets, barre de recherche et l'utilisateur peut accéder au fil de discussion correspondant en cliquant sur l'un des sujets.
- **Fil de discussion** (principale.jsx) : affichage du sujet, de ses commentaires, du formulaire de nouveau commentaire et des actions *répondre* / *supprimer* / *profil*.
- **Création de sujet** (PostForum.jsx) : formulaire de publication d'un nouveau sujet.
- **Profil** (Profile.jsx) : Les informations de l'utilisateur et son historique de ses messages.
- **Liste des utilisateurs** (ListUser.jsx) : recherche d'utilisateurs et accès à leur profil.

8.3. Appels au serveur

- App.jsx : vérifie l'authentification au chargement avec GET /api/user/me afin de choisir entre l'espace d'authentification et le tableau de bord.
- Login.jsx : envoie POST /api/user/connexion pour créer la session utilisateur.
- Register.jsx : envoie POST /api/user/inscription, puis POST /api/user/connexion pour connecter immédiatement l'utilisateur après l'inscription.
- Deconnexion.jsx : envoie POST /api/user/deconnexion qui supprime la session puis renvoie vers l'accueil.
- Forum.jsx : récupère les tous sujets avec GET /api/forum/sujet
- EventSource /api/forum/events : pour rafraîchir la liste lorsqu'un sujet ou d'un commentaire est créé ou supprimé.
- PostForum.jsx : envoie POST /api/forum/postforum pour créer un nouveau sujet.
- principale.jsx : charge le fil sélectionné avec GET /api/forum/getthread?sujetid=... et écoute aussi /api/forum/events pour mettre à jour les commentaires en temps réel.
- NewPostForm.jsx : envoie POST /api/forum/postthread pour ajouter un commentaire ou une réponse.

- `DeleteForum.jsx` : envoie DELETE `/api/forum/delete/sujet` ou DELETE `/api/forum/delete/thread` selon l'élément supprimé (un fil de discussion ou un commentaire).
- `Profile.jsx` : récupère soit un autre utilisateur avec GET `/api/user/{id}`, soit l'utilisateur courant, puis charge l'historique complet avec GET `/api/forum/getallthread` avant de filtrer les messages affichés.
- `ListUser.jsx` : récupère tous les utilisateurs avec GET `/api/user/alluser`.

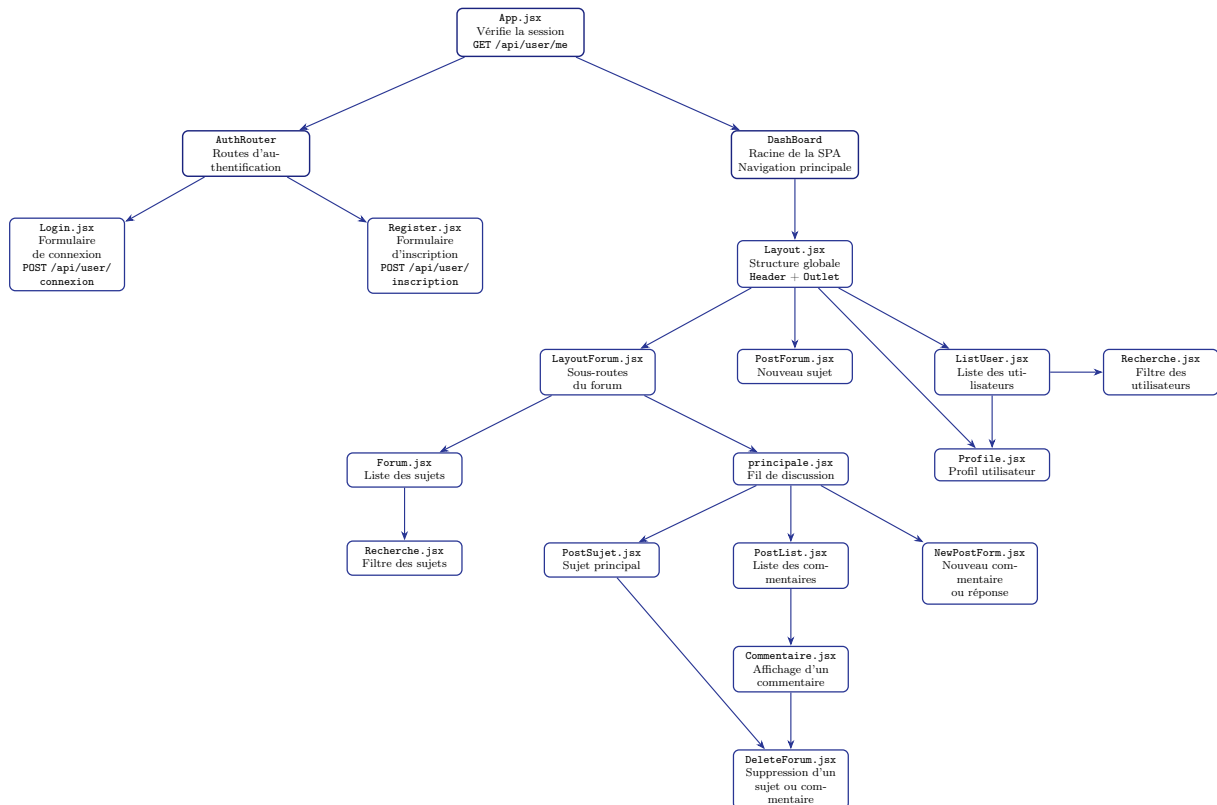


FIGURE 4 – Graphe du client React et de ses composants internes

9. Description des requêtes et réponses

Composant	Requête HTTP	Réponse JSON / contenu
App.jsx	GET /api/user/me avec le cookie session_token.	Objet User_db (id, pseudo, mdp, date, rang) ou code 401.
Login.jsx	POST /api/user/connexion avec un JSON contenant le pseudo et le mot de passe.	{"id", "message", "status"} en cas de succès, sinon {"Erreur"}.
Register.jsx	POST /api/user/inscription avec {"Pseudo", "Mdp", "Rang"} puis POST /api/user/connexion.	{"message": "Inscription réussie"} puis la réponse de connexion.
Deconnexion.jsx	POST /api/user/deconnexion.	{"message": "Déconnexion réussie"} ou {"message": "Déjà déconnecté"}.
Forum.jsx	GET /api/forum/sujet puis GET /api/forum/events pour le rafraîchissement temps réel.	Liste de Sujet_db (sujets avec id, titre, description, date, userid, userpseudo, prive) et événements SSE {"type", "action", "sujetId"}.
principale.jsx	GET /api/forum/getthread?sujet puis GET /api/forum/events.	Liste de Message_db (messages avec id, sujetid, content, date, userid, userpseudo, prive, repond) et événements SSE {"type", "action", "sujetId"/"threadId"}.
PostForum.jsx	POST /api/forum/postforum avec {"titre", "description", "userid", "userpseudo", "prive"}.	{"message": "Création du sujet réussie"}.
NewPostForm.jsx	POST /api/forum/postthread avec {"sujetid", "content", "userid", "userpseudo", "date", "prive", "repond"}.	{"message": "Création du message a réussie"}.
DeleteForum.jsx	DELETE /api/forum/delete/sujet ou DELETE /api/forum/delete/thread avec {"id"}.	{"message": "Suppresion du sujet/message a réussie"}.
Profile.jsx	GET /api/user/{id} ou GET /api/user/me, puis GET /api/forum/getallthread.	Objet User_db et liste de Message_db filtrée côté client selon le profil consulté.
ListUser.jsx	GET /api/user/alluser.	Tableau de User_db.

TABLE 5 – Résumé des échanges HTTP entre le client React et le serveur Go

10. Schéma global du système

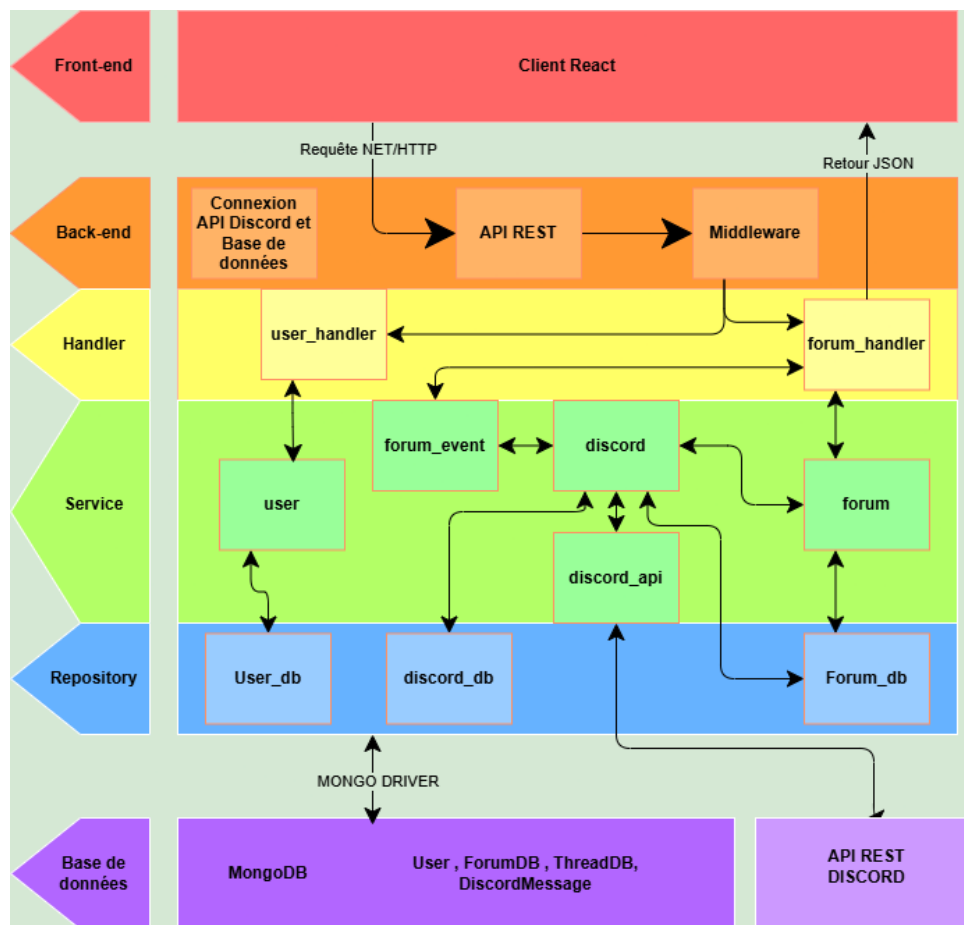


FIGURE 5 – Résumé des informations du dossier

11. Limites de conception, Problématiques et Solutions

L'architecture hybride et le choix d'une synchronisation périodique soulèvent plusieurs défis techniques. Cette section détaille les limitations identifiées lors de la conception, les solutions envisagées, ainsi que les pistes de réflexion autour de la sécurité et de la persistance.

11.1. Problèmes liés à la conception actuelle

- **Asymétrie des mises à jour et suppressions** : La récupération des messages reposant uniquement sur le dernier identifiant enregistré (`after={last_id}`), les modifications ou suppressions de messages effectuées a posteriori directement sur Discord ne sont pas répercutées sur le forum web. À l'inverse, les suppressions depuis le forum fonctionnent correctement car elles déclenchent immédiatement un événement **DELETE** vers l'API externe.
- **Latence de synchronisation et passage à l'échelle** : Le fait de vérifier régulièrement engendre une latence de 5 à 30 secondes. De plus, l'augmentation du nombre de fils de discussion (*threads*) accroît linéairement le nombre de requêtes à effectuer, ce qui peut impacter les performances globales du serveur.

- **Concurrence et verrous d'intégrité** : Quand plusieurs utilisateurs postent en même temps sur le forum, le système de verrous locaux (**Mutex**), se met en route pour garantir la cohérence lors de l'insertion et de la synchronisation. Mais cela génère des conflits d'accès qui ralentissent le traitement concurrent. Cette latence logicielle est une conséquence directe du choix de conception centralisé pour la récupération des données.
- **Gestion des formats de données** : L'application est actuellement limitée au format textuel brut. Les éléments les émojis, les images, les fichiers joints ainsi que certains caractères spéciaux comme `\n` ou `\s`, ne sont pas pris en compte pour le site web.

11.2. Analyse des solutions et arbitrages

Plusieurs pistes ont été explorées pour pallier ces limitations, menant à des choix d'ingénierie stricts :

- **Téléchargement intégral et tri global** : Une solution consistait à récupérer l'intégralité de l'historique Discord à chaque cycle pour effectuer une comparaison avec la base de données. Cette idée a été abandonnée car elle déplace le problème : le volume croissant de données aurait provoqué une latence ou une surcharge de requête discord (on est limité en nombre de requête par seconde).
- **Vérification d'existence par échantillonnage** : Émettre une requête périodique pour chaque message stocké en base de données afin de vérifier sa présence sur Discord s'est avéré être une mauvaise approche. Cette méthode sature instantanément les quotas de requêtes (*Rate Limits*) imposés par Discord en créant une boucle d'appels excessivement lourde.
- **Restriction des privilèges Discord (Solution retenue)** : Pour aligner le comportement des utilisateurs Discord avec les capacités actuelles du forum, les permissions du serveur Discord ont été restreintes. Les utilisateurs n'ont plus la possibilité d'envoyer de liens, d'images, de fichiers ou d'émojis complexes, garantissant l'homogénéité du contenu textuel.

11.3. Perspectives : Protection et Optimisation de la Base de Données

Pour faire évoluer l'application, deux axes majeurs de sécurité et de restructuration sont envisagés :

- **Durcissement de la sécurité des requêtes** : Actuellement, il est nécessaire de systématiser la vérification d'état pour chaque requête sensible. Chaque appel client devrait transmettre son identifiant de session afin que le serveur valide la correspondance exacte entre l'ID de l'utilisateur actif et les données de session stockées dans MongoDB avant d'autoriser l'accès ou la modification de la ressource.

12. Utilisation de l'Intelligence Artificielle

Dans le cadre de ce projet, l'Intelligence Artificielle (IA gemini) a été intégrée comme un outil d'accompagnement et un tuteur méthodologique tout au long du développement. Son usage s'est articulé autour de plusieurs axes majeurs :

- **Apprendre le langage** : N'ayant pas une maîtrise approfondie du langage Go et de la bibliothèque React, l'IA m'a permis d'avoir un support d'apprentissage interactif. Elle a permis de mieux comprendre la syntaxe et les concepts théoriques abordés en cours de PC3R.
- **Exploration technique de l'API externe** : L'IA a facilité la prise en main de l'API REST de Discord en explicitant la structure de ses ressources JSON et en guidant la configuration initiale du bot.
- **Aide au débogage et diagnostic** : Face aux erreurs générées par le compilateur Go ou l'environnement d'exécution dans le terminal, l'IA a été sollicitée pour analyser les traces de pile (*stack traces*) et cela a permis de comprendre les causes des erreurs.
- **Optimisation de l'interface (CSS)** : Sur la partie Front-end, elle a fourni une assistance précieuse pour structurer les feuilles de style, ajuster le rendu visuel de l'application web monopage et assurer une mise en page fluide des composants du forum.
- **Soutien rédactionnel** : Enfin, l'IA a été un outil d'aide à la reformulation textuelle qui me permettait de restructurer ce rapport dans un style plus académique.

Loin de se substituer au travail de conception, l'IA a agi comme un collaborateur technique, accélérant la résolution des points de blocage tout en maximisant la compréhension des paradigmes de programmation concurrente et web.